

C++ PREP DAY-3

Rule of three

a basic function should have three function a destructor ,copy constructor, copy assignment operator

```
class vec{
    int *ptr;
    size_t size;
public:
    vec() :ptr(nullptr), size(0){}
    ~vec() {delete []ptr;}
    vec(const vec& vec2){ // copy constructor
        ptr=new int[vec2.size]; // create size first
        size=vec2.size;// change the size
        std::copy(vec2.ptr,vec2.ptr+size,ptr); // the prev vec
        2 data to new pointer
    }
    vec& operator=(const vec& rhs){ // assignment operator
        vec veccopy(rhs); // create a new vec via copy constructor
        veccopy.swap(*this); //swap the vec copy to *this
        return *this; // return this
        /*
        if(this==&rhs) return *this;
        delete [] ptr;
        ptr= new int[rhs.size];
        for(size_t i=0;i<rhs.size;i++){
            ptr[i]=rhs.ptr[i];
        }
        size=rhs.size;
        return *this;
        */
    }
};
```

```

        */ // another way to write assignment operator

    }

};

```

RAII (Resource Acquisition Is Initialization)

if we declare a pointer we should also delete it due to some reasons we return before executing delete then a memory leak occurs {bad_code}

solution {writing our own custom pointer }

```

struct raii_ptr{
    int *ptr;
    raii_ptr(int *p){
        ptr=p;
    }
    ~raii_ptr() {
        delete [] ptr;
    }
    raii_ptr(const raii_ptr&)=delete;
    raii_ptr& operator=(const raii_ptr&)=delete;
};

```

this raii_ptr is non copyable and non deletable

instead of making that function =delete if we make that =default it assumes the actual version of it

Rule of Zero

the compiler can build all the three required by its we shouldn't write anything the compiler takes care of everything if we don't write any function

if we use `vector<int>` in the class then we don't need explicitly write all custom functions the compiler will do rest

Rule of Five

we should define the earlier three and move constructor, move assignment operator

```
class cusvec{
    int *ptr;
    size_t size;
public:
    cusvec(){
        ptr=nullptr;
        size=0;
    }
    cusvec(const cusvec & other){
        ptr=new int [other.size];
        size=other.size;
        for(size_t i=0;i<other.size;i++) ptr[i]=other.ptr[i];
    }
    cusvec& operator=(cusvec & other){
        if(this==&other) return *this;
        delete [] ptr;
        this->size=other.size;
        this->ptr= new int[other.size];
        for(size_t i=0;i<other.size;i++) this->ptr[i]=other.p
tr[i];
        return *this;
        /*
        cusvec temp(other);
        temp.swap(*this);
        return *this;
        */
    }
};
```

```

    }
    ~cusvec(){
        delete [] ptr ;
    }
    cusvec(cusvec &&other){
        ptr=other.ptr;
        size=other.size;
        other.ptr=nullptr;
        other.size=0;

    }
    cusvec& operator=(cusvec && other){
        if(this==&other) return *this;
        delete [] ptr;
        size=other.size;
        ptr=other.ptr;
        other.ptr=nullptr;
        other.size=0;
        return *this;
        /*
        cusvec temp(std :: move(other));
        temp.swap(*this);
        return *this;
        */
    }
};

```

Rule of Four (and a half)

so here we write the destructor ,copy constructor , move constructor ,and instead both assignment operators we only a single for both

where we just passing the assignemnter operator as it is

```

class cusvec{
    int *ptr;
    size_t size;
public:
    cusvec(){
        ptr=nullptr;
        size=0;
    }
    cusvec(const cusvec & other){
        ptr=new int [other.size];
        size=other.size;
        for(size_t i=0;i<other.size;i++) ptr[i]=other.ptr[i];
    }

    ~cusvec(){
        delete [] ptr ;
    }
    cusvec(cusvec &&other){
        ptr=other.ptr;
        size=other.size;
        other.ptr=nullptr;
        other.size=0;
    }
    cusvec& operator=(cusvec other){
        copy.swap(*this);
        return *this;
    }
    // here when we pass cusvec a=b copy is called
    // here whej we pass cusvec=move(b) then move is performe
d
    void swap(cusvec& other){
        using std :: swap;
        swap(ptr,other.ptr);
        swap(size,other.size);
    }
}

```

```
};
```

Examples of Resource Management

`unique_ptr` manages a raw pointer to a uniquely owned heap allocation.

- **Destructor** frees the resource
 - Calls `delete` on the raw pointer
- **Copy constructor** copies the resource
 - Copying doesn't make sense. We `=delete` this member function.
- **Move constructor** transfers ownership of the resource
 - Transfers the raw pointer, then nulls out the right-hand side
- **Copy assignment operator** frees the left-hand resource and copies the right-hand one
 - Copying doesn't make sense. We `=delete` this member function.
- **Move assignment operator** frees the left-hand resource and transfers ownership of the right-hand one
 - Calls `delete` on the left-hand ptr, transfers, then nulls out the right-hand ptr

60

`shared_ptr` manages a reference count.

- **Destructor** frees the resource
 - Decrements the refcount (and maybe cleans up if the refcount is now zero)
- **Copy constructor** copies the resource
 - Increments the refcount
- **Move constructor** transfers ownership of the resource
 - Leaves the refcount the same, then disengages the right-hand side
- **Copy assignment operator** frees the left-hand resource and copies the right-hand one
 - Decrements the old refcount, increments the new refcount
- **Move assignment operator** frees the left-hand resource and transfers ownership of the right-hand one
 - Decrements the old refcount, then disengages the right-hand side

6

unique_lock manages a lock on a mutex.

- **Destructor** frees the resource
 - Unlocks the mutex if "engaged"
- **Copy constructor** copies the resource
 - Copying doesn't make sense. We =delete this member function.
- **Move constructor** transfers ownership of the resource
 - Leaves the mutex in the same state, then disengages the right-hand side
- **Copy assignment operator** frees the left-hand resource and copies the right-hand one
 - Copying doesn't make sense. We =delete this member function.
- **Move assignment operator** frees the left-hand resource and transfers ownership of the right-hand one
 - Unlocks the old mutex if "engaged"; then transfers ownership from the right-hand side

62

Unique_pointer_syntax

this object has exactly one owner

```
std:: unique_ptr<int >p(new int(10));
// when this p dies the delete function is called automatical
ly
std :: unique_ptr<int >p2=p ; // not possible
std :: unique_ptr<int >p2=std :: move(p) ;// this is possible
//move construcor
std :: unique_ptr<int >p2(std:: move (p)) ;// this is also po
ssible
// move assignment is also possible
std :: unique_ptr<int >p3(new int(20)) ;
std :: unique_ptr<int >p4(new int(10));
p3=std :: move(p4);
// more ways to create unique pointer
auto p5=std:: make_unique_ptr<int >(10);
// release
int *np=p3.release();
//reset
p5.reset(20);//before it was 10 nnow it became 20
p5.swap(p4);
```

```
//for creating array
auto pa=make_unique_ptr<int []>(new int [5]);
```

Shared and weak Pointers Syntax

Smart Pointer	Owners Allowed
<code>unique_ptr</code>	Exactly 1 owner
<code>shared_ptr</code>	Multiple owners
<code>weak_ptr</code>	No ownership (observer only)

In shared pointer copying is allowed

so in shared pointer there can be many owners so to maintain that its has hidden integer stored by it which counts the number of users it have

when ref count becomes then the pointer is died untill then its alsive

```
// to see the number of owners
auto p1=std:: make_shared_ptr<int >(10);
auto p2=p1;
p1.use_count();// displyas the number of usesr (2) here
```

There is problem in using the shared poniter called CIRCULAR DIFFERENCE (SHARED POINTER CYCLE)

```
class A {
public:
    std::shared_ptr<B> ptr;
};
```



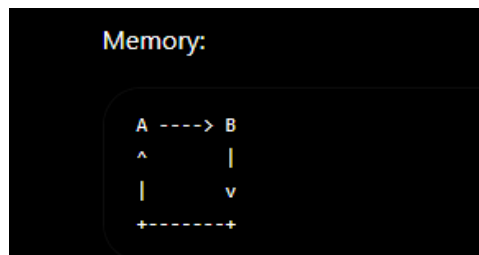
```

class B {
public:
    std::shared_ptr<A> ptr;
};

auto a = std::make_shared<A>();
auto b = std::make_shared<B>();

a->ptr = b;
b->ptr = a;

```



```

//after we make
a=nullptr;b=nullptr;
cout<<a.use_count()<<b.use_count();
// this will print 1,1 but actually it should be zero
//memory leak occurs

```

A shared_ptr cycle occurs when two or more objects own each other through a shared_ptr .since each share_ptr increases the ref count ,the count never gets to zero and the pointers never gets destroyed

To counter this shared pointer cycle weak pointer is introduce

Weak Pointer

when we copy a shared pointer to a weak pointer then the ref count doesnt increase

```
auto p = std::make_shared<int>(10);
```

```
std::weak_ptr<int> wp = p;
```

```
class A {  
public:  
    std::shared_ptr<B> ptr;  
};
```

```
class B {  
public:  
    std::weak_ptr<A> ptr;  
};  
// note for the same we used a weak_ptr
```

```
std::weak_ptr<int> wp;  
cout<<*wp;// this is illegal  
// weak pointer cant exist alone  
auto p=std::make_shared<int>(10);  
std::weak_ptr<int> wp=p;  
auto sp=wp.lock();// we must use lock to access the objects  
// the cout all the other things work  
cout<<*sp<<endl;
```